

# Symbiosis Institute of Technology, Pune

## Faculty of Engineering

## CSE- Academic Year 2026-27

## Java Programming – Lab Batch 2025-29

| Lab Assignment No:- 25 | |
|---|---|
| | |
| **Name of Student** | Arpit Chakraborty |
| **PRN No.** | 25070122047 |
| **Batch** | 25-29 |
| **Class** | CSE A2 |
| **Academic Year & Semester** | 26-27, III |
| **Date of Submission** | 27/10/26 |
| | |
| **Title of Assignment:** | Write a Java program using the Thread class to create multiple threads and demonstrate their lifecycle. |
| **Practice questions**<br><br>**Output Screenshots Solve questions on given link and attach screen shot of leetcode with your login and o/p** | **1.Create a program where multiple threads print numbers simultaneously**<br><br>class NumberThread extends Thread {<br>String name;<br><br>NumberThread(String name) {<br>this.name = name;<br>}<br><br>public void run() {<br>for (int i = 1; i <= 5; i++) {<br>System.out.println(name + ": " + i); |

```
            try {
                Thread.sleep(500);
            } catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}

public class ThreadDemo {
    public static void main(String[] args) {
        NumberThread t1 = new NumberThread("Thread 1");
        NumberThread t2 = new NumberThread("Thread 2");

        System.out.println("Thread 1 State: " + t1.getState());
        System.out.println("Thread 2 State: " + t2.getState());

        t1.start();
        t2.start();

        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            System.out.println(e);
        }

        System.out.println("Thread 1 State: " + t1.getState());
        System.out.println("Thread 2 State: " + t2.getState());
        System.out.println("Main thread completed.");
    }
}
```

**2. Create a Java program using multiple threads to simulate bank transactions. One thread performs deposit operations, and another performs withdrawal operations, both printing transaction details simultaneously**

```
class BankAccount {
    int balance = 1000;

    synchronized void deposit(int amount) {
        balance += amount;
        System.out.println("Deposited: " + amount);
        System.out.println("Current Balance: " + balance);
    }

    synchronized void withdraw(int amount) {
        if (balance >= amount) {
            balance -= amount;
            System.out.println("Withdrawn: " + amount);
            System.out.println("Current Balance: " + balance);
        } else {
            System.out.println("Insufficient balance!");
        }
    }
}

class DepositThread extends Thread {
    BankAccount account;

    DepositThread(BankAccount account) {
        this.account = account;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            account.deposit(100);
```

```
        }
    }
}

class WithdrawThread extends Thread {
    BankAccount account;

    WithdrawThread(BankAccount account) {
        this.account = account;
    }

    public void run() {
        for (int i = 1; i <= 5; i++) {
            account.withdraw(100);
        }
    }
}

public class BankTransaction {
    public static void main(String[] args) {
        BankAccount account = new BankAccount();

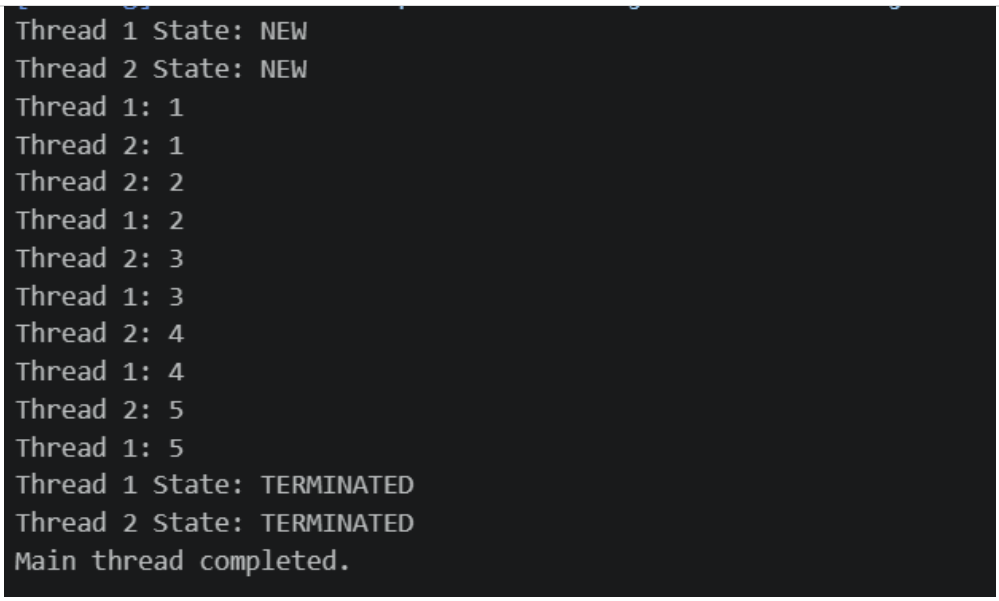
        DepositThread t1 = new DepositThread(account);
        WithdrawThread t2 = new WithdrawThread(account);

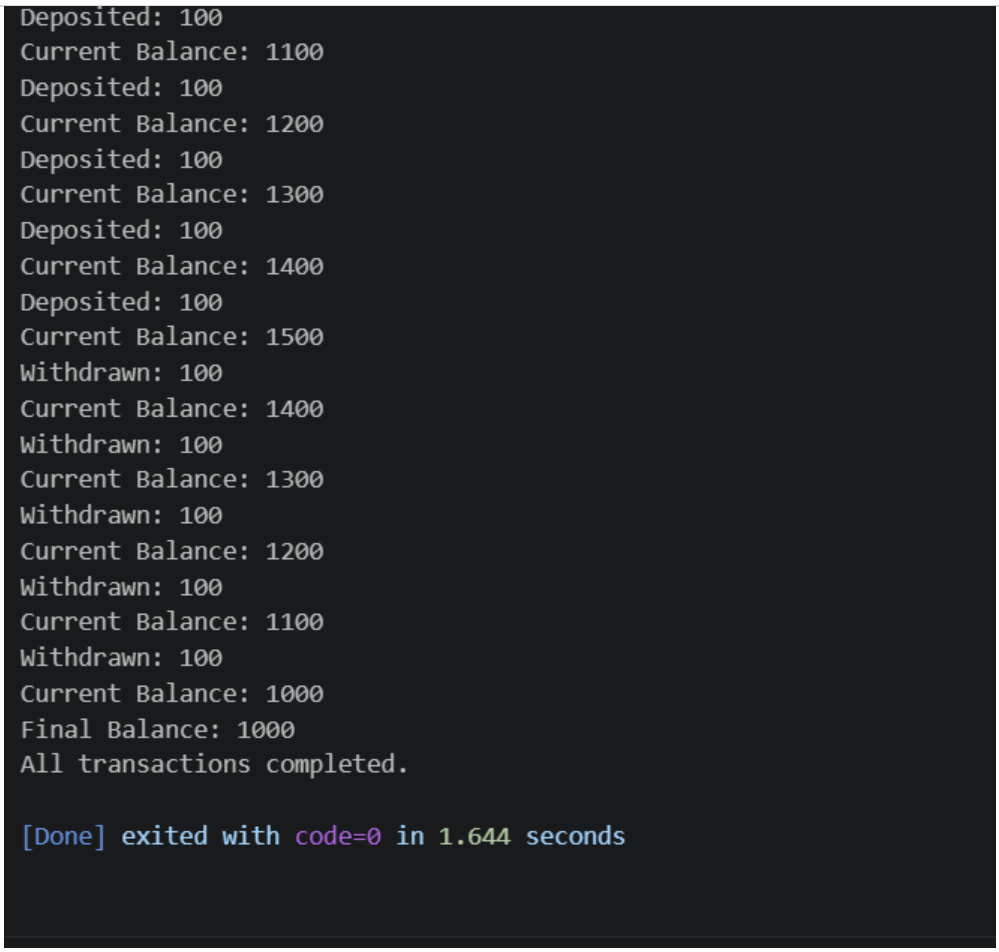
        t1.start();
        t2.start();

        try {
            t1.join();
            t2.join();
        } catch (InterruptedException e) {
            System.out.println(e);
        }
```

```
        System.out.println("Final Balance: " + account.balance);
        System.out.println("All transactions completed.");
    }
}
```

**Output Screenshots (if applicable)**



| | |
|---|---|
| |  |
| **Conclusion** | Thus, we successfully created and executed multiple threads using the Thread class in Java. We demonstrated thread lifecycle states and concurrent number printing, as well as bank deposit and withdrawal operations. We also learned how start(), sleep(), join(), and synchronized help manage threads and maintain data consistency. |